

VIT BHOPAL UNIVERSITY

PROJECT REPORT

Lost and Found Management System



Student Name	<u>Meet Thakkar</u>
Registration No.	<u>26BCE11339</u>
Programme / Branch	Computer Science / <u>Core CSE</u>
Course / Subject	<u>CSE1021</u>
Faculty / Mentor	<u>Venkat Prasadh Padhy</u>
Academic Year	2026–27

Submitted as part of the academic project work.

Abstract

The Lost and Found Management System is a simple Python-based console application designed to help users record, view, and manage information about lost and found items. The project was developed incrementally using basic Python concepts such as variables, lists, dictionaries, functions, conditional statements, loops, user input, and modular files. The application provides separate functionality for reporting lost items, reporting found items, and viewing the stored records. The project focuses on clear program flow and beginner-friendly implementation rather than advanced frameworks or databases.

Keywords: Python, Lost and Found, Console Application, Functions, Lists, Dictionaries, Modular Programming

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Scope of the Project
5	Technologies and Concepts Used
6	System Design and Working
7	Project Modules
8	Implementation Details
9	Testing and Results
10	Limitations
11	Future Enhancements
12	Conclusion
13	References

1. Introduction

Lost and found management is a common problem in schools, colleges, offices, and other public places. When an item is lost, the owner needs a simple way to describe it and later check whether someone has reported finding it. This project implements a small console-based system in Python to organize such information.

2. Problem Statement

Manual handling of lost and found information can become confusing when several items are reported. Important details such as item type, brand, colour, location, date, description, and status need to be recorded consistently. The project addresses this problem by providing a structured way to enter and display item records.

3. Objectives

The main objectives of the project are:

- Create a simple menu-driven Lost and Found application.
- Allow users to report lost items with relevant details.
- Allow users to report found items with relevant details.
- Store item records in a structured format using Python data structures.
- Display stored items in a readable format.
- Use separate Python files to keep related functions organized.
- Practice fundamental Python programming through a complete working project.

4. Scope of the Project

The current version is intended as a local, beginner-level console application. It focuses on entering and viewing lost/found records during program execution. It does not include a web interface, user authentication, online synchronization, or a permanent database. These can be added in future versions.

5. Technologies and Concepts Used

Technology / Concept	Purpose
Python	Main programming language
Lists	Store multiple item records
Dictionaries	Store details of an individual item
Functions	Divide the program into manageable operations
if / elif / else	Control program decisions and menu choices
Loops	Repeat menu and processing operations
input()	Collect information from the user

Technology / Concept	Purpose
Modules / multiple .py files	Organize lost, found, and view functionality

6. System Design and Working

The application follows a simple menu-driven flow. The main program presents options to the user. Depending on the selected option, the corresponding function is called. Lost-item operations, found-item operations, and viewing operations are separated into their own modules. Item information is stored using Python dictionaries inside a list, allowing multiple records to be handled during a program session.

User	Selects an option from the main menu
Main Program	Routes the request to the required function/module
Lost Module	Accepts and stores lost-item information
Found Module	Accepts and stores found-item information
View Module	Reads the stored records and displays them

7. Project Modules

Module	Responsibility
main.py	Runs the main program/menu and connects the different operations.
lost.py	Contains functionality for adding/reporting lost items.
found.py	Contains functionality for adding/reporting found items.
view.py	Contains functionality for displaying stored item information.

The exact filenames may be adjusted to match the final submitted folder structure. The important design principle is that lost, found, and viewing responsibilities are separated rather than being placed in one large file.

8. Implementation Details

8.1 Item Data

Each item is represented as a dictionary. Typical fields include an automatically assigned ID, report type (lost or found), item type, brand, colour, location, date, description, and status. A list named for the stored items can hold multiple dictionaries.

8.2 Item IDs

A numeric ID is used to identify individual records. A counter is increased when a new item is added, making each newly created record distinguishable during a program session.

8.3 Status

Items can carry a status such as active. This provides a foundation for later features such as marking an item as claimed, returned, or resolved.

8.4 Modular Structure

The original item-related functionality was divided into smaller files for lost items, found items, and viewing. This makes the code easier to navigate and demonstrates basic modular programming.

9. Testing and Results

The program was tested manually through its console interface. Different item details and menu operations were entered to verify that the application accepted input and displayed the expected records. During development, issues such as indentation and output formatting were corrected and the program was repeatedly run after changes.

Test Case	Input / Action	Expected Result
1	Run the program	Main menu is displayed
2	Add a lost item	Lost-item details are accepted and a record is created
3	Add a found item	Found-item details are accepted and a record is created
4	View items	Stored records are displayed in readable form
5	Add multiple records	Multiple item dictionaries are stored in the list
6	Use menu choices	Correct operation is selected based on user input

10. Limitations

- Data is stored in memory and is not automatically saved to a database or file for permanent storage.
- The application is console-based and does not have a graphical or web interface.
- There is no login or user authentication system.
- There is no automatic matching algorithm between lost and found reports.
- Input validation is basic and can be expanded for production use.

11. Future Enhancements

- Add file handling or a database such as SQLite for permanent storage.
- Add search and filtering by item type, brand, location, date, or status.
- Implement matching between lost and found reports.
- Add functionality to update an item's status after it is returned.
- Build a graphical interface or web application.
- Add authentication and role-based access for students and administrators.
- Add image upload support for better identification of items.

12. Conclusion

The Lost and Found Management System successfully demonstrates how fundamental Python programming concepts can be combined to build a complete, practical application. The project provides separate operations for lost and found reports and a viewing mechanism for stored records. More importantly, developing the project in small steps helped establish an understanding of functions, dictionaries, lists, conditions, loops, and modular code organization. The current version provides a useful foundation for adding persistent storage, searching, matching, and a graphical or web interface in the future.

13. References

- Python Software Foundation. Python Documentation — <https://docs.python.org/3/>
- Python Software Foundation. Python Tutorial — <https://docs.python.org/3/tutorial/>
- Course/class notes and practical exercises used during project development.

— End of Report —